

1878. Get Biggest Three Rhombus Sums in a Grid

Solved ✓

Medium

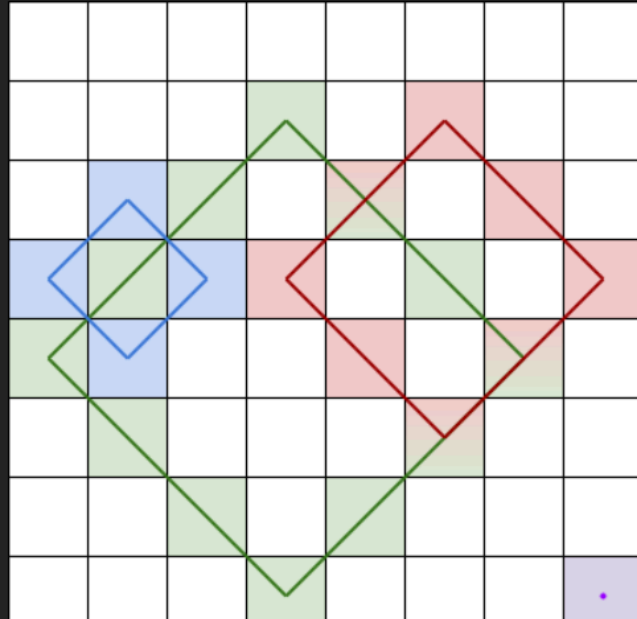
Topics

Companies

Hint

You are given an $m \times n$ integer matrix `grid`.

A **rhombus sum** is the sum of the elements that form **the border** of a regular rhombus shape in `grid`. The rhombus must have the shape of a square rotated 45 degrees with each of the corners centered in a grid cell. Below is an image of four valid rhombus shapes with the corresponding colored cells that should be included in each **rhombus sum**:



Note that the rhombus can have an area of 0, which is depicted by the purple rhombus in the bottom right corner.

Return *the biggest three **distinct rhombus sums** in the `grid` in **descending order**. If there are less than three distinct values, return all of them.*

Example 1:

3	4	5	1	3
3	3	4	2	3
20	30	200	40	10
1	5	5	4	1
4	3	2	2	5

Input: `grid = [[3,4,5,1,3],[3,3,4,2,3],[20,30,200,40,10],[1,5,5,4,1],[4,3,2,2,5]]`

Output: `[228,216,211]`

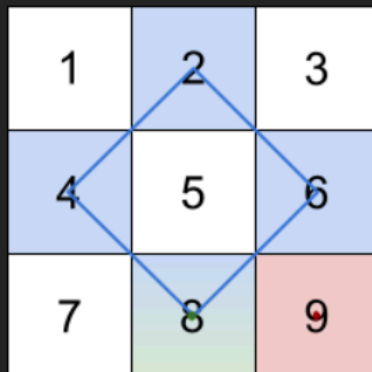
Explanation: The rhombus shapes for the three biggest distinct rhombus sums are depicted above.

– Blue: $20 + 3 + 200 + 5 = 228$

– Red: $200 + 2 + 10 + 4 = 216$

– Green: $5 + 200 + 4 + 2 = 211$

Example 2:



Input: `grid = [[1,2,3],[4,5,6],[7,8,9]]`

Output: `[20,9,8]`

Explanation: The rhombus shapes for the three biggest distinct rhombus sums are depicted above.

- Blue: $4 + 2 + 6 + 8 = 20$
- Red: 9 (area 0 rhombus in the bottom right corner)
- Green: 8 (area 0 rhombus in the bottom middle)

Example 3:

Input: `grid = [[7,7,7]]`

Output: `[7]`

Explanation: All three possible rhombus sums are the same, so return `[7]`.

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 50`
- `1 <= grid[i][j] <= 105`

Python:

class Solution:

def getBiggestThree(self, grid: List[List[int]]) -> List[int]:

 m, n = len(grid), len(grid[0])

 max_length = min((m - 1) // 2, (n - 1) // 2)

 best = []

 heapify(best)

 def rhombusSum(r, c, k):

 result = grid[r][c]

```

    for s in range(k):
        r += 1
        result += grid[r][c + s + 1]
        result += grid[r][c - s - 1]
    for s in range(k - 1, -1, -1):
        r += 1
        result += grid[r][c + s]
        if s > 0:
            result += grid[r][c - s]
    return result
for r in range(m):
    for c in range(n):
        for k in range(max_length + 1):
            if c >= k and c + k < n and r + 2 * k < m:
                candidate = rhombusSum(r, c, k)
                if candidate not in set(best):
                    heappush(best, candidate)
                if len(best) > 3:
                    heappop(best)
return sorted(best, reverse = True)

```

JavaScript:

```

var getBiggestThree = function(grid) {

    const m = grid.length;
    const n = grid[0].length;

    const set = new Set();

    for(let i=0;i<m;i++){
        for(let j=0;j<n;j++){

            set.add(grid[i][j]);

            for(let k=1;;k++){
                if(i-k<0 || i+k>=m || j-k<0 || j+k>=n) break;

                let sum = 0;

                let r=i-k, c=j;
                for(let t=0;t<k;t++) sum += grid[r+t][c+t];

                r=i, c=j+k;
                for(let t=0;t<k;t++) sum += grid[r+t][c-t];
            }
        }
    }
}

```

```

        r=i+k; c=j;
        for(let t=0;t<k;t++) sum += grid[r-t][c-t];

        r=i; c=j-k;
        for(let t=0;t<k;t++) sum += grid[r-t][c+t];

        set.add(sum);
    }
}

return Array.from(set).sort((a,b)=>b-a).slice(0,3);
};

```

Java:

```

class Solution {
    public int[] getBiggestThree(int[][] grid) {
        int m=grid.length;
        int n=grid[0].length;
        TreeSet<Integer> ts=new TreeSet<>((a,b)->Integer.compare(b,a));
        for(int i=0;i<m;i++){
            for(int j=0;j<n;j++){
                ts.add(grid[i][j]);
                for(int size=1;size<=Math.min(m,n);size++){
                    int sum=0;
                    boolean valid=true;
                    int x1=i,y1=j,x2=i,y2=j;
                    int dir1=-1,dir2=1;
                    int temp=0;
                    while(temp<=2*size){
                        if(Math.min(x1,x2)<0||Math.max(x1,x2)>=m||Math.min(y1,y2)<0||Math.max(y1,y2)>=n){
                            valid=false;
                            break;
                        }
                        if(x1==x2&&y1==y2){
                            sum+=grid[x1][y1];
                        }else{
                            sum+=grid[x1][y1];
                            sum+=grid[x2][y2];
                        }
                        if(temp==size){
                            dir2=dir2*dir1;
                        }
                    }
                }
            }
        }
    }
}

```

```

        dir1=dir2*dir1;
    }
    x1--;
    x2--;
    y1=y1+dir1;
    y2=y2+dir2;
    temp++;
}
if(valid){
    ts.add(sum);
}
}
}
}
List<Integer> lst=new ArrayList<>();
while(!ts.isEmpty()&&lst.size()<3){
    lst.add(ts.getFirst());
    ts.removeFirst();
}
return lst.stream().mapToInt(Integer::intValue).toArray();
}
}

```